

Week 5. 모듈화 (network / compute, env 조합)

개념 오늘의 목표

Week4까지의 코드를 재사용 가능한 모듈로 쪼갬다. `modules/network` (VPC·서브넷)와 `modules/compute` (EC2)를 만들고, `envs/prod` 에서 조립한다. 모듈은 입력(variable) → 처리(resource) → 출력(output)의 블랙박스다.

```
envs/prod/main.tf
├─ module "network" (in: cidr,subnets  out: vpc_id, subnet_ids)
└─ module "compute" (in: subnet_id      out: instance_id)
    ▲ network의 output을 compute의 input으로 연결
```

실습 1. network 모듈

`week5/modules/network/variables.tf` :

```
variable "project" { type = string }
variable "vpc_cidr" { type = string }
variable "subnets" {
  type = map(object({ cidr = string, az = string }))
}
```

`week5/modules/network/main.tf` :

```
resource "aws_vpc" "this" {
  cidr_block = var.vpc_cidr
  tags       = { Name = "${var.project}-vpc" }
}
resource "aws_subnet" "this" {
  for_each      = var.subnets
  vpc_id        = aws_vpc.this.id
  cidr_block    = each.value.cidr
  availability_zone = each.value.az
  tags          = { Name = "${var.project}-${each.key}" }
}
```

`week5/modules/network/outputs.tf` :

```
output "vpc_id" { value = aws_vpc.this.id }
output "subnet_ids" { value = { for k, s in aws_subnet.this : k => s.id } }
```

실습 2. compute 모듈

`week5/modules/compute/variables.tf` :

```
variable "project" { type = string }
variable "subnet_id" { type = string }
```

week5/modules/compute/main.tf :

```
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter { name = "name", values = ["al2023-ami-*-x86_64"] }
}
resource "aws_instance" "this" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  subnet_id     = var.subnet_id
  tags          = { Name = "${var.project}-web" }
}
```

week5/modules/compute/outputs.tf :

```
output "instance_id" { value = aws_instance.this.id }
```

실습 3. ★ env에서 모듈 조립

week5/envs/prod/main.tf :

```
terraform {
  required_version = ">= 1.9"
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
}
provider "aws" { region = "ap-northeast-2" }

module "network" {
  source  = "../../modules/network"
  project = "boaz-w5-prod"
  vpc_cidr = "10.0.0.0/16"
  subnets = {
    public-a = { cidr = "10.0.1.0/24", az = "ap-northeast-2a" }
    public-c = { cidr = "10.0.2.0/24", az = "ap-northeast-2c" }
  }
}

module "compute" {
  source      = "../../modules/compute"
  project     = "boaz-w5-prod"
  subnet_id   = module.network.subnet_ids["public-a"] # 모듈 간 연결 ★
}

output "web_instance_id" {
  value = module.compute.instance_id
}
```

실습 4. apply

```
cd week5/envs/prod
terraform init      # 모듈이 로드되며 "Initializing modules..." 표시
terraform apply
```

확인 모듈 구조 확인

- `init` 로그에 `- network in ../../modules/network, - compute in ...`
- `terraform state list` → 리소스 이름이 `module.network.aws_vpc.this` 처럼 모듈 경로 접두사를 가짐

관찰 🕒 기록

- state list에서 모듈 접두사를 가진 리소스 하나 적기: `module.____`
- `compute`가 `network`의 어떤 output을 입력으로 받았나: `__`

함정 Module not installed / source 경로 오류

모듈 코드를 새로 추가·경로 변경했으면 `terraform init` 을 다시 돌려야 인식된다. 상대경로(`../../..`) 깊이 확인.

정리 destroy 체크리스트

```
terraform destroy      # envs/prod 에서 실행하면 모든 모듈 리소스 삭제
```

- ☐ `network·compute` 리소스 전부 삭제
- ☐ 모듈은 코드일 뿐, 삭제 대상은 env가 만든 실제 리소스